Magnate Backend API Documentation

None

08 - Roberta Williams - Equipo de backend

2026

Tabla de Contenidos

1. Welcome to Magnate backend documentation 3
2. Integration 4
2.1 WebSockets guide (placeholder) 4
2.2 Actions 5
2.3 Responses 14
2.4 Fantasy events 18
3. Technical reference 21
3.1 Core Game Logic (games.py) 21

1. Welcome to Magnate backend documentation

2. Integration

2.1 WebSockets guide (placeholder)

URL de Conexión: `ws://backend.url/ws/game/{game_id}/`

2.1.1 Mensajes del Cliente (Frontend -> Backend)

Comando	Parámetros	Descripción
`roll_dice`	`{}`	Lanza los dados para el turno actual.
`buy_property`	`{"property_id": int}`	Compra la propiedad en la que está el jugador.

2.1.2 Mensajes del Servidor (Backend -> Frontend)

Evento	Payload	Descripción
`player_moved`	`{"player": id, "position": int}`	Notifica que un jugador se ha movido.
`error_occurred`	`{"message": str}`	Notifica un error de lógica (ver sección de Excepciones).

2.2 Actions

2.2.1 `Action`

Bases: `Model`

Represents generic action, so every action shares these fields

Frontend Request Payload Example: { "type": "Action", "game": 1, "player": 2, }

- **Source code in** `magnate/models.py`

```
class Action(models.Model):
    """
    Represents generic action, so every action shares these fields

    Frontend Request Payload Example:
    {
      "type": "Action",
      "game": 1,
      "player": 2,
    }
    """
    game = models.ForeignKey('Game', on_delete=models.CASCADE, related_name='in_game')
    player = models.ForeignKey('CustomUser', on_delete=models.CASCADE, related_name='made_by')
```

2.2.2 `ActionThrowDices`

Bases: `Action`

Action to throw the dices. It is basically empty.

Frontend Request Payload Example: { "type": "ActionThrowDices", "game": 1, "player": 2, }

- **Source code in** `magnate/models.py`

```
class ActionThrowDices(Action):
    """
    Action to throw the dices. It is basically empty.

    Frontend Request Payload Example:
    {
      "type": "ActionThrowDices",
      "game": 1,
      "player": 2,
    }
    """
    pass
```

2.2.3 `ActionMoveTo`

Bases: `Action`

Action to move to a square.

Frontend Request Payload Example: { "type": "ActionThrowDices", "game": 1, "player": 2, "square": 101, }

- **Source code in** `magnate/models.py`

```
class ActionMoveTo(Action):
    """
    Action to move to a square.

    Frontend Request Payload Example:
    {
      "type": "ActionThrowDices",
      "game": 1,
      "player": 2,
      "square": 101,
    }
    """
    square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='move_to')
```

2.2.4 `ActionTakeTram`

Bases: `Action`

Action to take the tram when possible

Frontend Request Payload Example: { "type": "ActionTakeTram", "game": 1, "player": 2, "square": 200, }

- **Source code in** `magnate/models.py`

```
class ActionTakeTram(Action):
    """
    Action to take the tram when possible

    Frontend Request Payload Example:
    {
      "type": "ActionTakeTram",
      "game": 1,
      "player": 2,
      "square": 200,
    }
    """
    square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='tram_move_to')
```

2.2.5 `ActionDropPurchase`

Bases: `Action`

Action to decline purchasing a property.

Frontend Request Payload Example: { "type": "ActionDropPurchase", "game": 1, "player": 2, "square": 15 }

- **Source code in** `magnate/models.py`

```
class ActionDropPurchase(Action):
    """
    Action to decline purchasing a property.

    Frontend Request Payload Example:
    {
      "type": "ActionDropPurchase",
      "game": 1,
      "player": 2,
      "square": 15
    }
    """
    square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='dropped')
```

2.2.6 `ActionBuySquare`

Bases: `Action`

Action to buy an unowned property.

Frontend Request Payload Example: { "type": "ActionBuySquare", "game": 1, "player": 2, "square": 15 }

- **Source code in** `magnate/models.py`

```
class ActionBuySquare(Action):
    """
    Action to buy an unowned property.

    Frontend Request Payload Example:
    {
      "type": "ActionBuySquare",
      "game": 1,
      "player": 2,
      "square": 15
    }
    """
    square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='bought')
```

2.2.7 `ActionSellSquare`

Bases: `Action`

Action to sell a property.

Frontend Request Payload Example: { "type": "ActionSellSquare", "game": 1, "player": 2, "square": 15 }

- **Source code in** `magnate/models.py`

```
class ActionSellSquare(Action):
    """
```

```
    Action to sell a property.

    Frontend Request Payload Example:
    {
      "type": "ActionSellSquare",
      "game": 1,
      "player": 2,
      "square": 15
    }
    """
    square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='sold')
```

2.2.8 `ActionBuild`

Bases: `Action`

Action to build houses/hotels on a property.

Frontend Request Payload Example: { "type": "ActionBuild", "game": 1, "player": 2, "houses": 1, "square": 12 }

- **Source code in** `magnate/models.py`

```
class ActionBuild(Action):
    """
    Action to build houses/hotels on a property.

    Frontend Request Payload Example:
    {
      "type": "ActionBuild",
      "game": 1,
      "player": 2,
      "houses": 1,
      "square": 12
    }
    """
    houses = models.IntegerField(default=1)
    square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='build_square')
```

2.2.9 `ActionDemolish`

Bases: `Action`

Action to demolish houses/hotels on a property.

Frontend Request Payload Example: { "type": "ActionDemolish", "game": 1, "player": 2, "houses": 1, "square": 12 }

- **Source code in** `magnate/models.py`

```
class ActionDemolish(Action):
    """
    Action to demolish houses/hotels on a property.

    Frontend Request Payload Example:
```

```
    {
      "type": "ActionDemolish",
      "game": 1,
      "player": 2,
      "houses": 1,
      "square": 12
    }
    """
    houses = models.IntegerField(default=0)
    square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='demolish_square')
```

2.2.10 `ActionChooseCard`

Bases: `Action`

Action to interact with or draw a fantasy card.

Frontend Request Payload Example: { "type": "ActionChooseCard", "game": 1, "player": 2, "chosen_revealed_card": true }

- **Source code in** `magnate/models.py`

```
class ActionChooseCard(Action):
    """
    Action to interact with or draw a fantasy card.

    Frontend Request Payload Example:
    {
      "type": "ActionChooseCard",
      "game": 1,
      "player": 2,
      "chosen_revealed_card": true
    }
    """
    chosen_revealed_card = models.BooleanField(default=False)
```

2.2.11 `ActionSurrender`

Bases: `Action`

Action to quit/surrender the game. It is empty.

Frontend Request Payload Example: { "type": "ActionSurrender", "game": 1, "player": 2 }

- **Source code in** `magnate/models.py`

```
class ActionSurrender(Action):
    """
    Action to quit/surrender the game. It is empty.

    Frontend Request Payload Example:
    {
      "type": "ActionSurrender",
```

```
      "game": 1,
      "player": 2
    }
    """
    pass
```

2.2.12 `ActionTradeProposal`

Bases: `Action`

Action to propose a trade to another player.

Frontend Request Payload Example: { "type": "ActionTradeProposal", "game": 1, "player": 2, "destination_user": 3, "offered_money": 200, "asked_money": 0, "offered_properties": [5, 6], "asked_properties": [12] }

- **Source code in** `magnate/models.py`

```
class ActionTradeProposal(Action):
    """
    Action to propose a trade to another player.

    Frontend Request Payload Example:
    {
      "type": "ActionTradeProposal",
      "game": 1,
      "player": 2,
      "destination_user": 3,
      "offered_money": 200,
      "asked_money": 0,
      "offered_properties": [5, 6],
      "asked_properties": [12]
    }
    """
    destination_user = models.ForeignKey('CustomUser', on_delete=models.CASCADE, related_name='destination_user')
    offered_money = models.PositiveIntegerField(default=0)
    asked_money = models.PositiveIntegerField(default=0)
    offered_properties = models.ManyToManyField('PropertyRelationship', related_name='offered_properties')
    asked_properties = models.ManyToManyField('PropertyRelationship', related_name='asked_properties')
```

2.2.13 `ActionTradeAnswer`

Bases: `Action`

Action to accept or decline a trade proposal.

Frontend Request Payload Example: { "type": "ActionTradeAnswer", "game": 1, "player": 2, "choose": true, "proposal": 8 }

- **Source code in** `magnate/models.py`

```
class ActionTradeAnswer(Action):
    """
```

```
    Action to accept or decline a trade proposal.

    Frontend Request Payload Example:
    {
      "type": "ActionTradeAnswer",
      "game": 1,
      "player": 2,
      "choose": true,
      "proposal": 8
    }
    """
    choose = models.BooleanField(default=False)
    proposal = models.OneToOneField('ActionTradeProposal', on_delete=models.CASCADE, related_name='proposal')
```

2.2.14 `ActionMortgageSet`

Bases: `Action`

Action to mortgage a property.

Frontend Request Payload Example: { "type": "ActionMortgageSet", "game": 1, "player": 2, "square": 15 }

- **Source code in** `magnate/models.py`

```
class ActionMortgageSet(Action):
    """
    Action to mortgage a property.

    Frontend Request Payload Example:
    {
      "type": "ActionMortgageSet",
      "game": 1,
      "player": 2,
      "square": 15
    }
    """
    square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='mortgage_set_square')
```

2.2.15 `ActionMortgageUnset`

Bases: `Action`

Action to lift a mortgage from a property.

Frontend Request Payload Example: { "type": "ActionMortgageUnset", "game": 1, "player": 2, "square": 15 }

- **Source code in** `magnate/models.py`

```
class ActionMortgageUnset(Action):
    """
    Action to lift a mortgage from a property.

    Frontend Request Payload Example:
```

```
    {
      "type": "ActionMortgageUnset",
      "game": 1,
      "player": 2,
      "square": 15
    }
    """
    square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='mortgage_unset_square')
```

2.2.16 `ActionPayBail`

Bases: `Action`

Action to pay the bail fee to get out of jail. It is empty.

Frontend Request Payload Example: { "type": "ActionPayBail", "game": 1, "player": 2 }

- **Source code in** `magnate/models.py`

```
class ActionPayBail(Action):
    """
    Action to pay the bail fee to get out of jail. It is empty.

    Frontend Request Payload Example:
    {
      "type": "ActionPayBail",
      "game": 1,
      "player": 2
    }
    """
    pass
```

2.2.17 `ActionNextPhase`

Bases: `Action`

Action to transition to the next game phase or end turn. It is empty.

Frontend Request Payload Example: { "type": "ActionNextPhase", "game": 1, "player": 2 }

- **Source code in** `magnate/models.py`

```
class ActionNextPhase(Action):
    """
    Action to transition to the next game phase or end turn. It is empty.

    Frontend Request Payload Example:
    {
      "type": "ActionNextPhase",
      "game": 1,
      "player": 2
    }
```

```
    """
    pass
```

2.2.18 `ActionBid`

Bases: `Action`

Action to place a bid on a property auction.

Frontend Request Payload Example: { "type": "ActionBid", "game": 1, "player": 2, "auction": 4, "amount": 150 }

- **Source code in** `magnate/models.py`

```
class ActionBid(Action):
    """
    Action to place a bid on a property auction.

    Frontend Request Payload Example:
    {
      "type": "ActionBid",
      "game": 1,
      "player": 2,
      "auction": 4,
      "amount": 150
    }
    """
    auction = models.ForeignKey('Auction', on_delete=models.CASCADE, related_name='bids', null=True, blank=True)
    amount = models.PositiveIntegerField(default=0)
```

2.3 Responses

2.3.1 `Response`

Bases: `Model`

Base model for game state updates broadcasted to the frontend.

Frontend Response Payload Example: { "type": "Response", "money": {"1": 1500, "2": 1200}, "active_phase_player": 2, "active_turn_player": 2, "phase": "management" }

- **Source code in** `magnate/models.py`

```
class Response(models.Model):
    """
    Base model for game state updates broadcasted to the frontend.

    Frontend Response Payload Example:
    {
      "type": "Response",
      "money": {"1": 1500, "2": 1200},
      "active_phase_player": 2,
      "active_turn_player": 2,
      "phase": "management"
    }
    """
    money = models.JSONField(default=dict, blank=True)
    active_phase_player = models.ForeignKey('CustomUser', on_delete=models.SET_NULL, null=True, related_name='respons
    active_turn_player = models.ForeignKey('CustomUser', on_delete=models.SET_NULL, null=True, related_name='response
    phase = models.CharField(choices=Game.GamePhase, max_length=20)
    positions = models.JSONField(default=dict, blank=True)
```

2.3.2 `ResponseMovement`

Bases: `Response`

Base response for an event that moves a player across the board.

Frontend Response Payload Example: { "type": "ResponseMovement", "money": {"1": 1500, "2": 1200}, "active_phase_player": 2, "active_turn_player": 2, "phase": "moving", "path": [10, 11, 12, 13, 14, 15], "fantasy_event": null }

- **Source code in** `magnate/models.py`

```
class ResponseMovement(Response):
    """
    Base response for an event that moves a player across the board.

    Frontend Response Payload Example:
    {
      "type": "ResponseMovement",
      "money": {"1": 1500, "2": 1200},
      "active_phase_player": 2,
      "active_turn_player": 2,
      "phase": "moving",
```

```
      "path": [10, 11, 12, 13, 14, 15],
      "fantasy_event": null
    }
    """
    path = models.JSONField(default=list, blank=True)
    fantasy_event = models.ForeignKey('FantasyEvent', on_delete=models.CASCADE, null=True, blank=True)
```

2.3.3 `ResponseThrowDices`

Bases: `ResponseMovement`

Response detailing the result of a dice roll and valid movement destinations.

Frontend Response Payload Example: { "type": "ResponseThrowDices", "money": {"1": 1500, "2": 1200}, "active_phase_player": 2, "active_turn_player": 2, "phase": "moving", "path": [10, 11, 12], "fantasy_event": null, "dice1": 4, "dice2": 3, "dice_bus": 1, "destinations": [17], "triple": false, "streak": 0 }

- **Source code in** `magnate/models.py`

```
class ResponseThrowDices(ResponseMovement):
    """
    Response detailing the result of a dice roll and valid movement destinations.

    Frontend Response Payload Example:
    {
      "type": "ResponseThrowDices",
      "money": {"1": 1500, "2": 1200},
      "active_phase_player": 2,
      "active_turn_player": 2,
      "phase": "moving",
      "path": [10, 11, 12],
      "fantasy_event": null,
      "dice1": 4,
      "dice2": 3,
      "dice_bus": 1,
      "destinations": [17],
      "triple": false,
      "streak": 0
    }
    """
    dice1 = models.PositiveIntegerField(default=0)
    dice2 = models.PositiveIntegerField(default=0)
    dice_bus = models.PositiveIntegerField(default=0)
    destinations = models.JSONField(default=list, blank=True)
    triple = models.BooleanField(default=False)
    streak = models.IntegerField(default=0)
```

2.3.4 `ResponseChooseSquare`

Bases: `ResponseMovement`

Response confirming a player's movement to a specifically chosen square.

Frontend Response Payload Example: { "type": "ResponseChooseSquare", "money": {"1": 1500, "2": 1200}, "active_phase_player": 2, "active_turn_player": 2, "phase": "moving", "path": [10, 25], "fantasy_event": null }

- **Source code in** `magnate/models.py`

```
class ResponseChooseSquare(ResponseMovement):
    """
    Response confirming a player's movement to a specifically chosen square.

    Frontend Response Payload Example:
    {
      "type": "ResponseChooseSquare",
      "money": {"1": 1500, "2": 1200},
      "active_phase_player": 2,
      "active_turn_player": 2,
      "phase": "moving",
      "path": [10, 25],
      "fantasy_event": null
    }
    """
    pass
```

2.3.5 `ResponseChooseFantasy`

Bases: `Response`

Response delivering the result of a drawn Chance/Community Chest card.

Frontend Response Payload Example: { "type": "ResponseChooseFantasy", "money": {"1": 1600, "2": 1200}, "active_phase_player": 2, "active_turn_player": 2, "phase": "management", "fantasy_event": 5 }

- **Source code in** `magnate/models.py`

```
class ResponseChooseFantasy(Response):
    """
    Response delivering the result of a drawn Chance/Community Chest card.

    Frontend Response Payload Example:
    {
      "type": "ResponseChooseFantasy",
      "money": {"1": 1600, "2": 1200},
      "active_phase_player": 2,
      "active_turn_player": 2,
      "phase": "management",
      "fantasy_event": 5
    }
    """
    fantasy_event = models.ForeignKey('FantasyEvent', on_delete=models.CASCADE, null=True, blank=True)
```

2.3.6 `ResponseAuction`

Bases: `Response`

Response updating the state of an ongoing or completed auction. Note: The @property decorators will likely be serialized as regular fields.

Frontend Response Payload Example: { "type": "ResponseAuction", "money": {"1": 1150, "2": 1200}, "active_phase_player": 1, "active_turn_player": 2, "phase": "management", "auction": 12, "winner": 1, "final_amount": 350, "is_tie": false }

- **Source code in** `magnate/models.py`

```
class ResponseAuction(Response):
    """
    Response updating the state of an ongoing or completed auction.
    Note: The @property decorators will likely be serialized as regular fields.

    Frontend Response Payload Example:
    {
      "type": "ResponseAuction",
      "money": {"1": 1150, "2": 1200},
      "active_phase_player": 1,
      "active_turn_player": 2,
      "phase": "management",
      "auction": 12,
      "winner": 1,
      "final_amount": 350,
      "is_tie": false
    }
    """
    auction = models.OneToOneField('Auction', on_delete=models.CASCADE, related_name='response')

    @property
    def winner(self):
        return self.auction.winner

    @property
    def final_amount(self):
        return self.auction.final_amount

    @property
    def is_tie(self):
        return self.auction.is_tie
```

2.4 Fantasy events

2.4.1 `FantasyEvent`

Bases: `Model`

Fantasy events are triggered when a player lands on a fantasy square. One of these effects takes place:

- winPlainMoney: Grants a flat amount of money. Cost: 130 | Possible values: [20, 60, 120, 150, 200]
- winRatioMoney: Increases money by a percentage ratio. Cost: 500 | Possible values: [1, 2, 5, 10]
- losePlainMoney: Deducts a flat amount of money. Cost: 80 | Possible values: [40, 80, 120, 150, 200]
- loseRatioMoney: Deducts money by a percentage ratio. Cost: 30 | Possible values: [1%, 2%, 5%, 10%]
- breakOpponentHouse: Demolishes a house on an opponent's property. Cost: 150
- breakOwnHouse: Demolishes a house on the player's own property. Cost: 30
- shufflePositions: Moves all players to random squares. Cost: 50
- moveAnywhereRandom: Moves the player to a random square. Cost: 50
- moveOpponentAnywhereRandom: Moves a random opponent to a random square. Cost: 60
- shareMoneyAll: Gives a specific amount of the player's money to all opponents. Cost: 5 | Possible values: [20, 30, 50]
- freeHouse: Builds a free house on one of the player's properties. Cost: 80
- goToJail: Sends the player directly to jail. Cost: 25
- sendToJail: Sends a random opponent directly to jail. Cost: 80
- everybodyToJail: Sends all players to jail. Cost: 50
- doubleOrNothing: 50/50 chance to either double current money or lose it all. Cost: 50
- getParkingMoney: Awards the accumulated parking money to the player. Cost: 500
- reviveProperty: Unmortgages a property for free (deshipotecas gratis). Cost: 100
- earthquake: Every developed property on the board loses one house (todas las calles pierden una casa). Cost: 200
- everybodySendsYouMoney: Every opponent pays a specific amount to the player. Cost: 120 | Possible values: [20, 30, 50]
- magnetism: Pulls all other players to the player's current square (todo el mundo va a ti). Cost: 100
- goToStart: Sends the player directly to the Start (Exit) square, collecting the pass-go money. Cost: 90

Frontend Fantasy Payload Example: { "type":"win_plain_money", "values":[20,60,120,150,200], "cost": 130 }

- **Source code in** `magnate/models.py`

```
class FantasyEvent(models.Model):
    """
    Fantasy events are triggered when a player lands on a fantasy square. One
    of these effects takes place:

    - winPlainMoney: Grants a flat amount of money.
        Cost: 130 | Possible values: [20, 60, 120, 150, 200]
    - winRatioMoney: Increases money by a percentage ratio.
        Cost: 500 | Possible values: [1, 2, 5, 10]
    - losePlainMoney: Deducts a flat amount of money.
        Cost: 80 | Possible values: [40, 80, 120, 150, 200]
    - loseRatioMoney: Deducts money by a percentage ratio.
        Cost: 30 | Possible values: [1%, 2%, 5%, 10%]
    - breakOpponentHouse: Demolishes a house on an opponent's property.
        Cost: 150
    - breakOwnHouse: Demolishes a house on the player's own property.
```

```
    Cost: 30
- shufflePositions: Moves all players to random squares.
    Cost: 50
- moveAnywhereRandom: Moves the player to a random square.
    Cost: 50
- moveOpponentAnywhereRandom: Moves a random opponent to a random square.
    Cost: 60
- shareMoneyAll: Gives a specific amount of the player's money to all opponents.
    Cost: 5 | Possible values: [20, 30, 50]
- freeHouse: Builds a free house on one of the player's properties.
    Cost: 80
- goToJail: Sends the player directly to jail.
    Cost: 25
- sendToJail: Sends a random opponent directly to jail.
    Cost: 80
- everybodyToJail: Sends all players to jail.
    Cost: 50
- doubleOrNothing: 50/50 chance to either double current money or lose it all.
    Cost: 50
- getParkingMoney: Awards the accumulated parking money to the player.
    Cost: 500
- reviveProperty: Unmortgages a property for free (deshipotecas gratis).
    Cost: 100
- earthquake: Every developed property on the board loses one house (todas las calles pierden una casa).
    Cost: 200
- everybodySendsYouMoney: Every opponent pays a specific amount to the player.
    Cost: 120 | Possible values: [20, 30, 50]
- magnetism: Pulls all other players to the player's current square (todo el mundo va a ti).
    Cost: 100
- goToStart: Sends the player directly to the Start (Exit) square, collecting the pass-go money.
    Cost: 90

Frontend Fantasy Payload Example:
{
  "type":"win_plain_money",
  "values":[20,60,120,150,200],
  "cost": 130
}
"""
class FantasyType(models.TextChoices):
    winPlainMoney = 'winPlainMoney',
    winRatioMoney = 'winRatioMoney',
    losePlainMoney = 'losePlainMoney',
    loseRatioMoney = 'loseRatioMoney',
    breakOpponentHouse = 'breakOpponentHouse',
    breakOwnHouse = 'breakOwnHouse',
    shufflePositions = 'shufflePositions',
    moveAnywhereRandom = 'moveAnywhereRandom',
    moveOpponentAnywhereRandom = 'moveOpponentAnywhereRandom',
    shareMoneyAll = 'shareMoneyAll',
    freeHouse = 'freeHouse',
    goToJail = 'goToJail',
    sendToJail = 'sendToJail',
    everybodyToJail = 'everybodyToJail',
    doubleOrNothing = 'doubleOrNothing',
    getParkingMoney = 'getParkingMoney',
    reviveProperty = 'reviveProperty',
    earthquake = 'earthquake',
    everybodySendsYouMoney = 'everybodySendsYouMoney',
```

```
        magnetism = 'magnetism',
        goToStart = 'goToStart'


    fantasy_type = models.CharField(choices=FantasyType, max_length=40)
    values = models.JSONField(null=True)
    card_cost = models.IntegerField(default=0)
```

3. Technical reference

3.1 Core Game Logic (games.py)

This module encapsulates the rules, state transitions, and actions of the game. It functions as the authoritative backend engine, ensuring that all moves are valid according to the game state.

3.1.1 🎲 Game State Machine Flow

The game operates as a finite state machine where players transition through specific phases. Any action sent by the frontend must be authorized by the `GameManager` based on the current active phase.

3.1.2 🛠️ Utility & Data Functions

These functions handle the database-level interactions required to resolve game logic.

Core Game Logic Module.

This module handles the rules, state transitions, and actions of the game. It provides helper functions to calculate net worth, rent, building/demolishing rules, and a GameManager class that acts as a state machine for the different phases of a player's turn.

3.1.3 `_get_square_by_custom_id(custom_id)`

Retrieves a BaseSquare instance by its custom_id.

Parameters:

Name	Type	Description	Default
`custom_id`	`int`	The custom identifier of the square.	*required*

Returns:

Name	Type	Description
`BaseSquare`	`BaseSquare`	The square instance.

Raises:

Type	Description
`GameLogicError`	If no square with the given custom_id exists.

- **Source code in** `magnate/game_utils.py`

```
def _get_square_by_custom_id(custom_id: int) -> BaseSquare:
    """
    Retrieves a BaseSquare instance by its custom_id.

    Args:
        custom_id (int): The custom identifier of the square.

    Returns:
        BaseSquare: The square instance.

    Raises:
```

```
        GameLogicError: If no square with the given custom_id exists.
    """
    square = BaseSquare.objects.filter(custom_id=custom_id).first()
    if square is None:
        raise GameLogicError(f"no square with id {custom_id}")

    return square
```

3.1.4 `_get_user_square(game, user)`

Retrieves the square where a specific user is currently located.

Parameters:

Name	Type	Description	Default
`game`	`Game`	The current game instance.	*required*
`user`	`CustomUser`	The user whose position is being queried.	*required*

Returns:

Name	Type	Description
`BaseSquare`	`BaseSquare`	The square where the user is currently standing.

Raises:

Type	Description
`GameLogicError`	If the user is not part of the game.

- **Source code in `magnate/game_utils.py`**

```
def _get_user_square(game: Game, user: CustomUser) -> BaseSquare:
    """
    Retrieves the square where a specific user is currently located.

    Args:
        game (Game): The current game instance.
        user (CustomUser): The user whose position is being queried.

    Returns:
        BaseSquare: The square where the user is currently standing.

    Raises:
        GameLogicError: If the user is not part of the game.
    """
    user_key = str(user.pk)

    if user_key not in game.positions:
        raise GameLogicError(f"user {user} not in the game")

    return _get_square_by_custom_id(game.positions[user_key])
```

3.1.5 `_get_relationship(game, square)`

Gets the ownership relationship between a game and a specific square.

Parameters:

Name	Type	Description	Default
`game`	`Game`	The current game instance.	*required*
`square`	`BaseSquare`	The property square to check.	*required*

Returns:

Type	Description
`Optional[PropertyRelationship]`	Optional[PropertyRelationship]: The relationship object if owned, None otherwise.

Raises:

Type	Description
`GameLogicError`	If more than one owner is found for the same square.

- **Source code in** `magnate/game_utils.py`

```
def _get_relationship(game: Game, square: BaseSquare) -> Optional[PropertyRelationship]:
    """
    Gets the ownership relationship between a game and a specific square.

    Args:
        game (Game): The current game instance.
        square (BaseSquare): The property square to check.

    Returns:
        Optional[PropertyRelationship]: The relationship object if owned, None otherwise.

    Raises:
        GameLogicError: If more than one owner is found for the same square.
    """
    try:
        return PropertyRelationship.objects.get(game=game, square=square)
    except PropertyRelationship.DoesNotExist:
        return None
    except MultipleObjectsReturned:
        raise GameLogicError("more than one owners for the same square")
```

3.1.6 `_get_jail_square()`

Retrieves the designated Jail square for the board.

Returns:

Name	Type	Description
`BaseSquare`	`BaseSquare`	The JailSquare instance.

Raises:

Type	Description
`GameDesignError`	If there are no jail squares or too many jail squares.

- **Source code in** `magnate/game_utils.py`

```
def _get_jail_square() -> BaseSquare:
    """
    Retrieves the designated Jail square for the board.

    Returns:
        BaseSquare: The JailSquare instance.

    Raises:
        GameDesignError: If there are no jail squares or too many jail squares.
    """
    try:
        return JailSquare.objects.get()
    except JailSquare.DoesNotExist:
        raise GameDesignError("there are no jail squares in the game")
    except MultipleObjectsReturned:
        raise GameDesignError("there are too many jail squares")
```

3.1.7 🏠 Property & Economy Rules

Calculates financial impacts, building constraints, and mortgage status.

Core Game Logic Module.

This module handles the rules, state transitions, and actions of the game. It provides helper functions to calculate net worth, rent, building/demolishing rules, and a GameManager class that acts as a state machine for the different phases of a player's turn.

3.1.8 `_calculate_rent_price(game, user, square)`

Calculates the rent price a user must pay when landing on a square.

Handles different logic for PropertySquares (houses), TramSquares, BridgeSquares, and ServerSquares based on ownership and multipliers.

Parameters:

Name	Type	Description	Default
`game`	`Game`	The current game instance.	*required*
`user`	`CustomUser`	The user landing on the square.	*required*
`square`	`BaseSquare`	The square being landed on.	*required*

Returns:

Name	Type	Description
`int`	`int`	The calculated rent amount to pay. Returns 0 if unowned or owned by the user.

Raises:

Type	Description
`GameDesignError`	If the rent arrays on the square are incorrectly configured.
`GameLogicError`	If an unexpected condition occurs during calculation.

- **Source code in** `magnate/game_utils.py`

```
def _calculate_rent_price(game: Game, user: CustomUser, square: BaseSquare) -> int:
    """
    Calculates the rent price a user must pay when landing on a square.

    Handles different logic for PropertySquares (houses), TramSquares,
    BridgeSquares, and ServerSquares based on ownership and multipliers.

    Args:
        game (Game): The current game instance.
        user (CustomUser): The user landing on the square.
        square (BaseSquare): The square being landed on.

    Returns:
        int: The calculated rent amount to pay. Returns 0 if unowned or owned by the user.

    Raises:
        GameDesignError: If the rent arrays on the square are incorrectly configured.
        GameLogicError: If an unexpected condition occurs during calculation.
    """
    # If it is not owned or is owned by the same user, no rent is paid
    prop_rel = _get_relationship(game, square)
    if not prop_rel or prop_rel.owner == user or prop_rel.mortgage:
        return 0

    houses = prop_rel.houses

    if isinstance(square, PropertySquare):
        if not square.rent_prices or len(square.rent_prices) < 6:
            raise GameDesignError(f"Incorrect rent prices for square {square.custom_id}")
        if houses == -1:
            return square.rent_prices[0]
        elif houses == 0:
            return square.rent_prices[0] * 2  # grupo completo = doble del alquiler base
        elif houses == 1:
            return square.rent_prices[1]
        elif houses == 2:
            return square.rent_prices[2]
        elif houses == 3:
            return square.rent_prices[3]
        elif houses == 4:
            return square.rent_prices[4]
        elif houses == 5:  # hotel
            return square.rent_prices[5]

        return 0

    elif isinstance(square, TramSquare):
        # TODO
        return 0

    elif isinstance(square, BridgeSquare):
```

```
        property_owner = prop_rel.owner
        bridges_owned = PropertyRelationship.objects.filter(game=game, square__bridgesquare__isnull=False, owner=prop
        if not square.rent_prices or len(square.rent_prices) < bridges_owned:
            raise GameDesignError(f"Incorrect rent prices for bridge {square.custom_id}")
        return square.rent_prices[bridges_owned - 1]

    elif isinstance(square, ServerSquare):
        property_owner = prop_rel.owner

        if not square.rent_prices or len(square.rent_prices) < 2:
            raise GameDesignError(f"Incorrect rent prices for square {square.custom_id}")

        squares = PropertyRelationship.objects.filter(game=game, square__serversquare__isnull=False, owner=property_

        if squares.count() == 2:
            return square.rent_prices[1]
        elif squares.count() == 1:
            return square.rent_prices[0]
        else:
            # TODO: Write something
            raise GameLogicError()
    else:
        return 0
```

3.1.9 `_build_square(game, user, building_square, number_built, free_build)`

Builds houses or hotels on a property.

Validates that the user owns the complete color group and respects the uniform building rule.

Parameters:

Name	Type	Description	Default
`game`	`Game`	The current game instance.	*required*
`user`	`CustomUser`	The user attempting to build.	*required*
`building_square`	`BaseSquare`	The target property square.	*required*
`number_built`	`int`	Number of buildings to construct.	*required*
`free_build`	`bool`	If True, the user is not charged for the construction.	*required*

Returns:

Name	Type	Description
`PropertyRelationship`	`PropertyRelationship`	The updated ownership relationship.

Raises:

Type	Description
`MaliciousUserInput`	If the user does not own the full group, violates uniform building rules, or tries to build beyond the max limit.
`GameLogicError`	If negative house values are encountered.

- **Source code in** `magnate/game_utils.py`

```
def _build_square(game: Game,
                  user: CustomUser,
                  building_square: BaseSquare,
                  number_built: int,
                  free_build: bool) -> PropertyRelationship:
    """
    Builds houses or hotels on a property.

    Validates that the user owns the complete color group and respects
    the uniform building rule.

    Args:
        game (Game): The current game instance.
        user (CustomUser): The user attempting to build.
        building_square (BaseSquare): The target property square.
        number_built (int): Number of buildings to construct.
        free_build (bool): If True, the user is not charged for the construction.

    Returns:
        PropertyRelationship: The updated ownership relationship.

    Raises:
        MaliciousUserInput: If the user does not own the full group, violates
            uniform building rules, or tries to build beyond the max limit.
        GameLogicError: If negative house values are encountered.
    """
    # Check if it's  a property and take its group
    building_square = building_square.get_real_instance()

    if not isinstance(building_square, PropertySquare):
        raise MaliciousUserInput(user, "tried to build in a non property square")

    relationship = _get_relationship(game, building_square)

    if relationship is None:
        raise MaliciousUserInput(user, "no user owns this square")

    if relationship.owner != user:
        raise MaliciousUserInput(user, "tried to build in an unowned property")

    square_group = building_square.group
    actual_houses = relationship.houses

    # Check if user has every square in the group and if its a property
    total_squares_in_group = PropertySquare.objects.filter(
        board=building_square.board,
        group=square_group
    ).count()

    group_relationships = PropertyRelationship.objects.filter(
        game=game,
        owner=user,
        square__propertysquare__group=square_group
    ).select_related('square')

    if group_relationships.count() != total_squares_in_group:
        raise MaliciousUserInput(user, "does not own the group")
```

```
    for rel in group_relationships:
        if rel.houses < 0:
            raise GameLogicError(f"negative house value")
        elif actual_houses + number_built - 1 > rel.houses:
            raise MaliciousUserInput(user, "already owns more than other")

    if actual_houses == 5:
        raise MaliciousUserInput(user, "nothing more to build")

    relationship.houses += number_built
    relationship.save()

    stats = PlayerGameStatistic.objects.get(user=user,game=game)
    stats.built_houses += number_built

    if not free_build:
        coste = building_square.build_price * number_built

        game.money[str(user.pk)] -= coste
        game.save()
        stats.lost_money += coste

    stats.save()

    return relationship  #ack
```

3.1.10 `_demolish_square(game, user, demolition_square, number_demolished, free_demolish)`

Demolishes houses/hotels on a property and returns money to the user.

Ensures that uniform building rules are respected during demolition.

Parameters:

Name	Type	Description	Default
`game`	`Game`	The current game instance.	*required*
`user`	`CustomUser`	The user attempting to demolish.	*required*
`demolition_square`	`BaseSquare`	The square where buildings will be demolished.	*required*
`number_demolished`	`int`	The amount of houses to demolish.	*required*
`free_demolish`	`bool`	If True, no money is refunded to the user.	*required*

Returns:

Name	Type	Description
`PropertyRelationship`	`PropertyRelationship`	The updated ownership relationship.

Raises:

Type	Description
`MaliciousUserInput`	If the property is unowned, owned by someone else, is not a property square, or violates uniform building rules.

- **Source code in** `magnate/game_utils.py`

```
def _demolish_square(game: Game,
                     user: CustomUser,
                     demolition_square: BaseSquare,
                     number_demolished: int,
                     free_demolish: bool) -> PropertyRelationship:
    """
    Demolishes houses/hotels on a property and returns money to the user.

    Ensures that uniform building rules are respected during demolition.

    Args:
        game (Game): The current game instance.
        user (CustomUser): The user attempting to demolish.
        demolition_square (BaseSquare): The square where buildings will be demolished.
        number_demolished (int): The amount of houses to demolish.
        free_demolish (bool): If True, no money is refunded to the user.

    Returns:
        PropertyRelationship: The updated ownership relationship.

    Raises:
        MaliciousUserInput: If the property is unowned, owned by someone else,
            is not a property square, or violates uniform building rules.
    """
    # Check if it's a property
    demolition_square = demolition_square.get_real_instance()
    if not isinstance(demolition_square, PropertySquare):
        raise MaliciousUserInput(user, "tried to demolish a non property square")

    relationship = _get_relationship(game, demolition_square)

    if relationship is None:
        raise MaliciousUserInput(user, "no user owns this square")

    if relationship.owner != user:
        raise MaliciousUserInput(user, "tried to demolish an unowned property")

    actual_houses = relationship.houses

    if actual_houses < number_demolished:
        raise MaliciousUserInput(user, "tried to demolish more houses than are built")

    square_group = demolition_square.group

    group_relationships = PropertyRelationship.objects.filter(
        game=game,
        owner=user,
        square__propertysquare__group=square_group
    ).select_related('square')

    # Check if we can demolish -> respect rule
    for rel in group_relationships:
        if (actual_houses - number_demolished) < (rel.houses - 1):
            raise MaliciousUserInput(user, "unable to demolish so many houses: violates the uniform building rule")

    # demolish
    relationship.houses -= number_demolished
```

```
    relationship.save()

    stats = PlayerGameStatistic.objects.get(user=user,game=game)
    stats.demolished_houses += number_demolished

    if not free_demolish:
        coste = demolition_square.build_price

        game.money[str(user.pk)] += coste // 2 * number_demolished
        game.save()
        stats.won_money += coste // 2 * number_demolished

    stats.save()

    return relationship
```

3.1.11 `_set_mortgage(game, user, target_square, free_mortgage)`

Mortgages a property to receive immediate funds.

Parameters:

Name	Type	Description	Default
`game`	`Game`	The current game instance.	*required*
`user`	`CustomUser`	The user attempting the mortgage.	*required*
`target_square`	`BaseSquare`	The property to be mortgaged.	*required*
`free_mortgage`	`bool`	If True, no money is added to the user's balance.	*required*

Returns:

Name	Type	Description
`PropertyRelationship`	`PropertyRelationship`	The updated relationship.

Raises:

Type	Description
`MaliciousUserInput`	If not owned, already mortgaged, or wrong type of square.
`GameLogicError`	If trying to mortgage a property that still has houses built on it.

- **Source code in** `magnate/game_utils.py`

```
def _set_mortgage(game: Game, user: CustomUser, target_square: BaseSquare, free_mortgage: bool) -> PropertyRelationsh
    """
    Mortgages a property to receive immediate funds.

    Args:
        game (Game): The current game instance.
        user (CustomUser): The user attempting the mortgage.
        target_square (BaseSquare): The property to be mortgaged.
        free_mortgage (bool): If True, no money is added to the user's balance.

    Returns:
        PropertyRelationship: The updated relationship.

    Raises:
```

```
        MaliciousUserInput: If not owned, already mortgaged, or wrong type of square.
        GameLogicError: If trying to mortgage a property that still has houses built on it.
    """
    target_square = target_square.get_real_instance()
    if not (isinstance(target_square, PropertySquare) or
            isinstance(target_square, BridgeSquare) or
            isinstance(target_square, ServerSquare)):
        raise MaliciousUserInput(user, "tried to mortgage a non property/bridge/server square")

    relationship = _get_relationship(game=game, square=target_square)

    if relationship is None:
        raise MaliciousUserInput(user, "no user owns this square")

    if relationship.owner != user:
        raise MaliciousUserInput(user, "tried to mortgage an unowned property")

    if relationship.mortgage:
        raise MaliciousUserInput(user, "tried to mortgage an already mortgaged property")



    if isinstance(target_square, PropertySquare):
        if relationship.houses > 0:
            raise GameLogicError("tried to mortgage a property with houses")

    relationship.mortgage = True
    relationship.save()
    stats = PlayerGameStatistic.objects.get(user=user,game=game)
    stats.num_mortgages += 1

    if not free_mortgage:
        mortgage_value = target_square.buy_price // 2
        game.money[str(user.pk)] += mortgage_value

        stats.won_money += mortgage_value
        game.save()

    stats.save()

    return relationship
```

3.1.12 `_unset_mortgage(game, user, target_square, free_unset_mortgage)`

Lifts the mortgage from a property by paying the required fee.

Parameters:

Name	Type	Description	Default
`game`	`Game`	The current game instance.	*required*
`user`	`CustomUser`	The user lifting the mortgage.	*required*
`target_square`	`BaseSquare`	The property to unmortgage.	*required*
`free_unset_mortgage`	`bool`	If True, the user is not charged.	*required*

Returns:

Name	Type	Description
`PropertyRelationship`	`PropertyRelationship`	The updated relationship.

Raises:

Type	Description
`MaliciousUserInput`	If not owned, not mortgaged, or wrong type of square.

- **Source code in** `magnate/game_utils.py`

```
def _unset_mortgage(game: Game, user: CustomUser, target_square: BaseSquare, free_unset_mortgage: bool) -> PropertyRe
    """
    Lifts the mortgage from a property by paying the required fee.

    Args:
        game (Game): The current game instance.
        user (CustomUser): The user lifting the mortgage.
        target_square (BaseSquare): The property to unmortgage.
        free_unset_mortgage (bool): If True, the user is not charged.

    Returns:
        PropertyRelationship: The updated relationship.

    Raises:
        MaliciousUserInput: If not owned, not mortgaged, or wrong type of square.
    """
    target_square = target_square.get_real_instance()
    if not (isinstance(target_square, PropertySquare) or
            isinstance(target_square, BridgeSquare) or
            isinstance(target_square, ServerSquare)):
        raise MaliciousUserInput(user, "tried to unset mortgage a non property/bridge/server square")

    relationship = _get_relationship(game=game, square=target_square)

    if relationship is None:
        raise MaliciousUserInput(user, "no user owns this square")

    if relationship.owner != user:
        raise MaliciousUserInput(user, "tried to unset mortgage an unowned property")

    if not relationship.mortgage:
        raise MaliciousUserInput(user, "tried to unset mortgage a not mortgaged property")

    relationship.mortgage = False
    relationship.save()
    target_square = target_square.get_real_instance()

    if not free_unset_mortgage:
        mortgage_value = target_square.buy_price // 2
        game.money[str(user.pk)] -= mortgage_value
        game.save()
        stats = PlayerGameStatistic.objects.get(user=user,game=game)
        stats.lost_money += mortgage_value
```

```
    return relationship
```

3.1.13 ⚙️ GameManager

The primary gateway for all frontend actions. Every action sent via WebSocket or REST must be processed through the `process_action` method.

3.1.14 `magnate.games.GameManager`

State machine and core processor for Game Actions.

The GameManager encapsulates the logic for each specific phase of a player's turn, processing polymorphic Action objects and mutating the database state accordingly.

- **Source code in `magnate/games.py`**

```
class GameManager:
    """
    State machine and core processor for Game Actions.

    The GameManager encapsulates the logic for each specific phase of a
    player's turn, processing polymorphic Action objects and mutating the
    database state accordingly.
    """
    ##############################################################################
    # Phase logic
    ##############################################################################

    ROLL_THE_DICES = Game.GamePhase.roll_the_dices
    CHOOSE_SQUARE = Game.GamePhase.choose_square
    MANAGEMENT = Game.GamePhase.management
    BUSINESS = Game.GamePhase.business
    ANSWER_TRADE_PROPOSAL = Game.GamePhase.proposal_acceptance
    LIQUIDATION = Game.GamePhase.liquidation
    AUCTION = Game.GamePhase.auction
    PROPOSAL_ACCEPTANCE = Game.GamePhase.proposal_acceptance
    CHOOSE_FANTASY = Game.GamePhase.choose_fantasy
    END_GAME = Game.GamePhase.end_game


    @classmethod
    @database_sync_to_async
    def process_action(cls, game: Game, user: CustomUser, action: Action) -> Response:
        """
        The only public method exposed in the API. It processes each action
        in dedicated functions depending on the current phase and returns
        a Response.

        Args:
            game (Game): The current active game instance.
            user (CustomUser): The user performing the action.
            action (Action): The deserialized action payload.

        Returns:
```

```
            Response: A response object representing the outcome.

        Raises:
            MaliciousUserInput: If the user acts out of turn or phase.
            GameLogicError: If an unrecognized phase is encountered.
        """
        if isinstance(action, ActionSurrender):
            # TODO
            pass

        if user != game.active_phase_player and not isinstance(action, ActionBid): # if aucction there are no turns
            raise MaliciousUserInput(user, "is not the active player")

        if game.phase == cls.ROLL_THE_DICES:
            if isinstance(action, ActionPayBail):
                response = cls._pay_bail_logic(game, user, action)
            elif isinstance(action, ActionThrowDices):
                response = cls._roll_dices_logic(game, user, action)
            else:
                raise MaliciousUserInputAction(game, user, action)
        elif game.phase == cls.CHOOSE_SQUARE:
            if not isinstance(action, ActionMoveTo):
                raise MaliciousUserInputAction(game, user, action)
            response = cls._square_chosen_logic(game, user, action)
        elif game.phase == cls.CHOOSE_FANTASY:
            if not isinstance(action, ActionChooseCard):
                raise MaliciousUserInputAction(game, user, action)
            response = cls._choose_fantasy_logic(game, user, action)
        elif game.phase == cls.MANAGEMENT:
            if not isinstance(action, (ActionBuySquare, ActionDropPurchase, ActionTakeTram, ActionNextPhase)):
                raise MaliciousUserInputAction(game, user, action)
            response = cls._management_logic(game, user, action)
        elif game.phase == cls.BUSINESS or game.phase == cls.LIQUIDATION:
            if not isinstance(action, (ActionBuild, ActionDemolish, ActionTradeProposal, ActionMortgageSet, ActionMo
                raise MaliciousUserInputAction(game, user, action)
            response = cls._business_logic(game, user, action)
        elif game.phase == cls.ANSWER_TRADE_PROPOSAL:
            if not isinstance(action, ActionTradeAnswer):
                raise MaliciousUserInputAction(game, user, action)
            response = cls._answer_trade_proposal_logic(game, user, action)
        elif game.phase == cls.AUCTION:
            if not isinstance(action, ActionBid):
                raise MaliciousUserInputAction(game, user, action)
            response = cls._bid_property_auction_logic(game, user, action)
        elif game.phase == cls.END_GAME:
            # TODO: check instance???
            # response = cls._end_game_logic(game,user,action)
            return cls._end_game_logic(game,user,action)
        else:
            raise GameLogicError(f"Fase no reconocida o no manejada: {game.phase}")

        return _add_basic_response_data(game, response)

    @staticmethod
    def _pay_bail_logic(game: Game, user: CustomUser, action: ActionPayBail) -> Response:
        """
        Processes the action to pay the jail bail.

        Args:
```

```
        game (Game): The current game instance.
        user (CustomUser): The jailed user.
        action (ActionPayBail): The bail action payload.

    Returns:
        Response: The standard response.

    Raises:
        MaliciousUserInput: If the user is not actually in jail.
        GameLogicError: If the user does not have enough money.
    """
    square = _get_user_square(game, user).get_real_instance()

    if not isinstance(square, JailSquare):
        raise MaliciousUserInput(user, "is not in jail")

    remaining = game.jail_remaining_turns.get(str(user.pk), 0)
    if remaining == 0:
        raise MaliciousUserInput(user, "is not in jail (no turns remaining)")

    bail_price = square.bail_price

    if game.money[str(user.pk)] < bail_price:
        raise GameLogicError("not enough money to pay bail")

    game.money[str(user.pk)] -= bail_price
    game.jail_remaining_turns[str(user.pk)] = 0
    game.save()

    stats = PlayerGameStatistic.objects.get(user=user,game=game)
    stats.lost_money += bail_price
    stats.save()
    # roll the dices -> continue normally

    return Response()

@staticmethod
def _roll_dices_logic(game: Game, user: CustomUser, action: ActionThrowDices) -> Response:
    """
    Handles rolling the dice, checking for doubles/triples, and resolving jail mechanics.

    Args:
        game (Game): The current game instance.
        user (CustomUser): The user throwing the dice.
        action (ActionThrowDices): The dice throw action payload.

    Returns:
        Response: Standard response.
    """
    response: ResponseThrowDices = ResponseThrowDices()

    d1 = random.randint(1,6)
    d2 = random.randint(1,6)
    d3 = random.randint(1,6) # 4-6 are the bus faces

    response.dice1, response.dice2, response.dice_bus = d1, d2, d3

    triples = d3 <= 3 and (d1 == d2 == d3)
    doubles = (d1 == d2) and not triples
```

```
response.triple = triples

current_pos_square = _get_user_square(game, user).get_real_instance()
current_pos_id = current_pos_square.custom_id

# jail logic
remaining_jail_turns = game.jail_remaining_turns.get(str(user.pk), 0)
is_jailed = remaining_jail_turns > 0

if is_jailed:
    jail_sq = current_pos_square
    if isinstance(jail_sq, JailSquare):
        if remaining_jail_turns == 1: #obligado a salir
            game.money[str(user.pk)] -= jail_sq.bail_price
            game.jail_remaining_turns[str(user.pk)] = 0
            stats = PlayerGameStatistic.objects.get(user=user,game=game)
            stats.turns_in_jail += 1
            stats.lost_money += jail_sq.bail_price
            stats.save()
        elif doubles: #sale gratis
            game.jail_remaining_turns[str(user.pk)] = 0
            game.streak = 0
        else:
            # stays in jail
            game.jail_remaining_turns[str(user.pk)] -= 1
            game.phase = GameManager.BUSINESS
            game.save()
            stats = PlayerGameStatistic.objects.get(user=user,game=game)
            stats.turns_in_jail += 1
            stats.save()
            response.path = [current_pos_id]
            return response
    else:
        raise GameLogicError(f"Cannot be in jail status and not in jail square")

# Not jailed
if triples:
    # path current -> decided in chosen
    response.triple = True
    square = current_pos_square
    all_squares = BaseSquare.objects.filter(board=square.board)
    # All squares are suitable destinations
    possible_destinations = [s.custom_id for s in all_squares]
    possible_destinations.remove(_get_jail_square().custom_id)
    game.possible_destinations = {str(c_id): 0 for c_id in possible_destinations}
    response.destinations = possible_destinations
    game.phase = GameManager.CHOOSE_SQUARE
    response.path = [current_pos_id]
    game.save()
    return response
elif doubles: # doubles streak only if not getting out of jail via doubles
    if game.streak >= 2:
        # path -> current and jail
        jail_square = _get_jail_square()
        response.destinations = [jail_square.custom_id]
        game.streak = 0
        response.streak = game.streak
```

```
            game.positions[str(user.pk)] = jail_square.custom_id
            game.jail_remaining_turns[str(user.pk)] = 3
            response.path = [current_pos_id, jail_square.custom_id]

            stats = PlayerGameStatistic.objects.get(user=user,game=game)
            stats.times_in_jail += 1
            stats.save()

            game.phase = GameManager.LIQUIDATION

            game.save()
            return response
        elif not is_jailed:
            game.streak = game.streak + 1
    else:
        game.streak = 0

    response.streak = game.streak

    # Hasn't gone to jail
    dice_combinations = _compute_dice_combinations(d1, d2, d3)
    game.possible_destinations, passed_go_map = _get_possible_destinations_ids(game, user, dice_combinations)

    response.destinations = list(game.possible_destinations.keys())
    if len(game.possible_destinations) > 1:
        # path in square chosen logic
        game.phase = GameManager.CHOOSE_SQUARE
        response.path = [current_pos_id]
    else:
        stats = PlayerGameStatistic.objects.get(user=user,game=game)
        stats.walked_squares += dice_combinations[0]
        stats.save()
        dest_square_id = next(iter(game.possible_destinations))
        steps = game.possible_destinations[dest_square_id]

        # Use _move_player_logic to get the traversed path and check for "Go to Jail" or "Passing Go".
        move_result = _move_player_logic(current_pos_square, steps)
        response.path = move_result["path"]

        if move_result["jailed"]:
            # landing in go to jail; update state to jail the player.
            game.positions[str(user.pk)] = move_result["final_id"]
            game.jail_remaining_turns[str(user.pk)] = 3
            game.phase = GameManager.LIQUIDATION
            stats = PlayerGameStatistic.objects.get(user=user, game=game)
            stats.times_in_jail += 1
            stats.save()
        else:
            game.positions[str(user.pk)] = dest_square_id
            square = _get_square_by_custom_id(dest_square_id)
            _apply_square_arrival(game, user, response, square, move_result["passed_go"])


    game.save()
    return response

@staticmethod
def _square_chosen_logic(game: Game, user: CustomUser, action: Action) -> Response:
    """
```

```
        Handles the logic when a user selects their final destination (if multiple choices were given).

        Args:
            game (Game): The current game instance.
            user (CustomUser): The current user.
            action (ActionMoveTo): Action indicating the chosen square.

        Returns:
            Response: Standard response.

        Raises:
            MaliciousUserInput: If the chosen square is not in `possible_destinations`.
        """
        response: ResponseChooseSquare = ResponseChooseSquare()

        if not isinstance(action, ActionMoveTo):
            raise MaliciousUserInputAction(game, user, action)

        square = action.square

        if str(square.custom_id) not in game.possible_destinations:
            raise MaliciousUserInput(user, "tried to move to an illegal square")

        current_pos_id = game.positions[str(user.pk)]
        current_pos_square = _get_square_by_custom_id(current_pos_id).get_real_instance()

        steps = game.possible_destinations.get(str(square.custom_id), 0)

        move_result = _move_player_logic(current_pos_square, steps)

        if steps == 0:
            response.path = [current_pos_id, square.custom_id]
        else:
            response.path = move_result["path"]

        game.positions[str(user.pk)] = square.custom_id

        stats = PlayerGameStatistic.objects.get(user=user,game=game)
        stats.walked_squares += steps
        stats.save()

        game.possible_destinations = dict()

        # Use the passed_go flag correctly computed by _move_player_logic.
        _apply_square_arrival(game, user, response, action.square, move_result["passed_go"])


        game.save()
        return response

    @staticmethod
    def _choose_fantasy_logic(game: Game, user: CustomUser, action: ActionChooseCard) -> Response:
        response = ResponseChooseFantasy()
        fantasy_event = game.fantasy_event
        generate = not action.chosen_revealed_card
        new_fantasy = None

        if not generate:
            apply_fantasy_event(game, user, fantasy_event)
```

```
            response.fantasy_event = fantasy_event
            game.fantasy_event = None

        else:
            new_fantasy = FantasyEventFactory.generate()
            apply_fantasy_event(game, user, new_fantasy)
            response.fantasy_event = new_fantasy
            game.fantasy_event = None


        if game.streak == 0:
            game.phase = GameManager.BUSINESS
        else:
            game.phase = GameManager.ROLL_THE_DICES

        game.save()

        return response

    @staticmethod
    def _management_logic(game: Game, user: CustomUser, action: Action) -> Response:
        """
        Logic of management phase, where the user can buy properties, pay bills etc.

        It checks the user's action against the current square they are on to update
        the game state and transition to the next phase.

        Args:
            game (Game): The current game instance.
            user (CustomUser): The acting user.
            action (Action): The payload of the action taken (e.g., ActionBuySquare).

        Returns:
            Response: Standard response.

        Raises:
            MaliciousUserInputAction: If the action does not fit the phase/context.
        """
        current_square = _get_user_square(game, user).get_real_instance()
        prop_rel = _get_relationship(game, current_square)

        if isinstance(action, ActionBuySquare):
            if isinstance(current_square, PropertySquare):
                # TODO: Check money
                game.money[str(user.pk)] -= current_square.buy_price
                stats = PlayerGameStatistic.objects.get(user=user,game=game)
                stats.lost_money += current_square.buy_price
                stats.save()
                new_property = PropertyRelationship(game=game, square=current_square, owner=user)
                user_properties = PropertyRelationship.objects.filter(game=game, owner=user)
                user_same_group_properties = user_properties.filter(square__propertysquare__group=current_square.grou

                group_squares = PropertySquare.objects.filter(group=current_square.group, board = current_square.boar

                if user_same_group_properties.count() == group_squares.count() - 1:
                    new_property.houses = 0
                    user_same_group_properties.update(houses=0)
                else:
                    new_property.houses = -1
```

```
            new_property.save()

        elif isinstance(current_square, ServerSquare):
            game.money[str(user.pk)] -= current_square.buy_price
            stats = PlayerGameStatistic.objects.get(user=user,game=game)
            stats.lost_money += current_square.buy_price
            stats.save()
            new_property = PropertyRelationship(game=game, square=current_square, owner=user)
            new_property.save()

        elif isinstance(current_square, BridgeSquare):
            game.money[str(user.pk)] -= current_square.buy_price
            stats = PlayerGameStatistic.objects.get(user=user, game=game)
            stats.lost_money += current_square.buy_price
            stats.save()
            new_property = PropertyRelationship(game=game, square=current_square, owner=user)
            new_property.save()

        else:
            raise MaliciousUserInputAction(game, user, action)
    elif isinstance(action, ActionDropPurchase):
        if isinstance(action.square, (PropertySquare, ServerSquare, BridgeSquare)):
            GameManager._initiate_auction(game, action.square)
        else:
            raise MaliciousUserInputAction(game, user, action)
    elif isinstance(action, ActionTakeTram):
        if isinstance(current_square, TramSquare):
            square = action.square
            tram_squares = TramSquare.objects.filter()
            tram_square_actual_id = game.positions[str(user.pk)]
            tram_squares_extern_ids = [s.custom_id for s in tram_squares]

            if square.custom_id == tram_square_actual_id: # case stay in the same square, free
                pass
            elif square.custom_id in tram_squares_extern_ids: # Move to another square
                if game.money[str(user.pk)] < square.buy_price:
                    raise MaliciousUserInput(user, "does not have enough money to take tram")
                game.money[str(user.pk)] -= square.buy_price
                stats = PlayerGameStatistic.objects.get(user=user,game=game)
                stats.lost_money += square.buy_price
                stats.save()
                game.positions[str(user.pk)] = square.custom_id
            else:
                raise MaliciousUserInput(user, "tried to take a tram to a non tram square")
        else:
            raise MaliciousUserInputAction(game, user, action)
    elif isinstance(action, ActionNextPhase):
        # TODO: Remove?
        pass
    else:
        raise MaliciousUserInputAction(game, user, action)

    if game.phase == GameManager.MANAGEMENT:
        if game.streak == 0:
            game.phase = GameManager.BUSINESS
        else:
            game.phase = GameManager.ROLL_THE_DICES

    game.save()
```

```
        return Response()

    @staticmethod
    def _business_logic(game: Game, user: CustomUser, action: Action) -> Response:
        """
        Unifies the business and liquidation phases. Handles building, demolishing,
        trading, mortgaging, and proceeding to the next turn.

        Args:
            game (Game): The current game instance.
            user (CustomUser): The active user.
            action (Action): The payload of the action taken.

        Returns:
            Response: Standard response.

        Raises:
            MaliciousUserInput: If restrictions (e.g., negative balance on next turn) are unmet.
        """
        # Logic for the business phase where players can build houses, trade, etc.
        if isinstance(action, ActionBuild):
            # Check owner
            building_square = action.square
            relationship = _build_square(game, user, building_square, action.houses, False)

        elif isinstance(action, ActionDemolish):
            # Similiar to build
            demolition_square = action.square
            relationship = _demolish_square(game, user, demolition_square, action.houses, False)

        elif isinstance(action, ActionTradeProposal):
            relationship = GameManager._propose_trade(game, user, action)

        elif isinstance(action, ActionMortgageSet):
            relationship = _set_mortgage(game, user, action.square, False)

        elif isinstance(action, ActionMortgageUnset):
            relationship = _unset_mortgage(game, user, action.square, False)
        elif isinstance(action, ActionNextPhase):
            current_money = game.money[str(user.pk)]


            if game.phase == GameManager.BUSINESS:
                if current_money >= 0:
                    GameManager._next_turn(game, user)
                else:
                    game.phase = GameManager.LIQUIDATION
                    game.save()

            elif game.phase == GameManager.LIQUIDATION:
                if current_money >= 0:
                    GameManager._next_turn(game, user)
                else:
                    raise MaliciousUserInput(user, "Cannot end in NEGATIVE")

        #TODO: elif with a timeout -> auto sell in case user can reach positive status or kick out in case

        return Response()
```

```
    @staticmethod
    def _answer_trade_proposal_logic(game: Game, user: CustomUser, action: Action) -> Response:
        """
        Processes a user's answer (accept/reject) to an active trade proposal.

        Args:
            game (Game): The current game instance.
            user (CustomUser): The targeted user responding to the offer.
            action (ActionTradeAnswer): The user's response payload.

        Returns:
            Response: Standard response.

        Raises:
            MaliciousUserInput: If an unauthorized user attempts to answer,
                or references an invalid proposal.
        """
        if isinstance(action, ActionTradeAnswer):
            offer = action.proposal

            offering = offer.player

            offered_money = offer.offered_money
            asked_money = offer.asked_money
            offered_properties = offer.offered_properties
            asked_properties = offer.asked_properties

            # TODO: Verify no player goes to negative (unless liquidation)

            if user != offer.destination_user:
                raise MaliciousUserInput(user, f"cannot accept proposal {offer}")

            if offer != game.proposal:
                raise MaliciousUserInput(user, f"tried to reference a non-existent proposal")

            if action.choose:
                for relationship in offered_properties.all():
                    relationship.owner = user
                    relationship.houses = -1 # reset houses
                    relationship.save()

                for relationship in asked_properties.all():
                    relationship.owner = offering
                    relationship.houses = -1
                    relationship.save()

                final_money = offered_money - asked_money

                game.money[str(offering.pk)] -= offered_money
                game.money[str(offering.pk)] += asked_money

                stats = PlayerGameStatistic.objects.get(user=offering,game=game)
                if final_money > 0:
                    stats.lost_money += abs(final_money)
                else:
                    stats.won_money += abs(final_money)
                stats.num_trades += 1
                stats.save()
```

```
            game.money[str(user.pk)] -= asked_money
            game.money[str(user.pk)] += offered_money

            stats = PlayerGameStatistic.objects.get(user=user,game=game)
            if final_money > 0:
                stats.won_money += abs(final_money)
            else:
                stats.lost_money += abs(final_money)
            stats.num_trades += 1
            stats.save()

            game.save()

        game.phase = GameManager.BUSINESS
        game.active_phase_player = offering
        game.save()

        return Response()
    else:
        raise MaliciousUserInputAction(game, user, action)

@staticmethod
def _initiate_auction(game: Game, square: BaseSquare) -> Response:
    """
    Transitions the game into an AUCTION phase for a dropped property.

    Args:
        game (Game): The current game instance.
        square (BaseSquare): The property square going up for auction.

    Returns:
        Response: Standard response.
    """
    game.phase = GameManager.AUCTION

    auction = Auction.objects.create(game=game, square=square)
    game.current_auction = auction
    game.save()

    # TODO: Remove magic number
    from .tasks import auction_callback
    auction_callback.apply_async(args=[game.pk], countdown=50)
    # TODO: Remove in production
    game.refresh_from_db()

    return Response()

@staticmethod
def _bid_property_auction_logic(game: Game, user: CustomUser, action: Action) -> Response:
    """
    Registers a player's bid during an active auction phase.

    Args:
        game (Game): The current game instance.
        user (CustomUser): The user placing the bid.
        action (ActionBid): The action carrying the bid amount.

    Returns:
        Response: Standard response.
```

```
        Raises:
            GameLogicError: If auction state is corrupted.
            MaliciousUserInput: If the user bids twice or exceeds their balance.
        """
        if not isinstance(action, ActionBid):
            raise MaliciousUserInputAction(game, user, action)

        auction = game.current_auction
        if not auction:
            raise GameLogicError("No active auction")

        # user has not bid yet
        if auction.bids.filter(player=user).exists():
            raise MaliciousUserInput(user, "User already placed a bid in this auction")

        # user who started the bid cant bid
        dropped = ActionDropPurchase.objects.filter(game=game, player=user, square=auction.square).exists()
        if dropped:
            raise MaliciousUserInput(user, "cannot bid in an auction they triggered")

        # user has enough money -> compulsory for auctions
        amount = action.amount
        if amount > game.money[str(user.pk)]:
            raise MaliciousUserInput(user, "Bid amount exceeds current balance")


        game.save()
        return Response()

    @staticmethod
    def _end_auction(game: Game) -> Response:
        """
        Ends an active auction, resolves the winner based on the highest bid,
        handles ties, and transitions the game state back to BUSINESS.

        Args:
            game (Game): The current game instance.

        Returns:
            Auction: The finalized auction object.

        Raises:
            GameLogicError: If not in AUCTION phase or state is missing square ID.
        """
        ## call this with a timer -> end of the auction

        if game.phase != GameManager.AUCTION:
            raise GameLogicError("Tried to end auction but game is not in auction phase")

        auction = game.current_auction
        if not auction:
            raise GameLogicError("Game is in AUCTION phase but no auction object found")

        square = auction.square.get_real_instance()
        bids = auction.bids.all()


        # no one bid
```

```
        if not bids.exists():
            if game.streak == 0:
                game.phase = GameManager.BUSINESS
            else:
                game.phase = GameManager.ROLL_THE_DICES

            auction.is_active = False
            auction.winner = None
            auction.final_amount = 0
            auction.is_tie = False
            auction.save()

            game.current_auction = None
            game.save()

            return ResponseAuction(auction=auction)

        max_bid_amount = bids.aggregate(Max('amount'))['amount__max']
        winners = bids.filter(amount=max_bid_amount)

        if winners.count() > 1:
            if game.streak == 0:
                game.phase = GameManager.BUSINESS
            else:
                game.phase = GameManager.ROLL_THE_DICES

            auction.is_active = False
            auction.winner = None
            auction.final_amount = max_bid_amount
            auction.is_tie = True
            auction.save()

            game.current_auction = None
            game.save()
            return ResponseAuction(auction=auction)

        winner_action = winners.first()
        winner = winner_action.player
        highest_bid = winner_action.amount

        game.money[str(winner.pk)] -= highest_bid
        stats = PlayerGameStatistic.objects.get(user=winner,game=game)
        stats.lost_money += highest_bid
        stats.save()

        new_property = PropertyRelationship(game=game, square=square, owner=winner)

        # groups n houses
        if isinstance(square, PropertySquare):
            user_properties = PropertyRelationship.objects.filter(game=game, owner=winner)
            user_same_group_properties = user_properties.filter(square__propertysquare__group=square.group)
            group_squares = PropertySquare.objects.filter(group=square.group, board = square.board)

            if user_same_group_properties.count() == group_squares.count() - 1:
                new_property.houses = 0
                user_same_group_properties.update(houses=0)
            else:
                new_property.houses = -1
        else:
```

```
            new_property.houses = -1

        new_property.save()

        auction.winner = winner
        auction.final_amount = highest_bid
        auction.is_active = False
        auction.is_tie = False
        auction.save()

        if game.streak == 0:
            game.phase = GameManager.BUSINESS
        else:
            game.phase = GameManager.ROLL_THE_DICES

        game.current_auction = None
        game.save()

        return ResponseAuction(auction=auction)

    @classmethod
    def _next_turn(cls, game: Game, user: CustomUser) -> None:
        players_list = list(game.players.all())
        num_players = len(players_list)
        print(f"players list: {players_list}")
        print(f"ordered players: {game.ordered_players}")
        current_index = -1
        current_player_id = -1
        for p in players_list:
            if p == game.active_turn_player:
                current_player_id = p.pk
                current_index = game.ordered_players.index(current_player_id)
                break

        if current_index == -1:
            raise GameLogicError('current player not found')

        next_index = (current_index + 1) % num_players
        print(f"next index: {next_index}")
        # The next active user is for both: phase and turn
        next_player = game.players.filter(pk=game.ordered_players[next_index]).first()
        if next_player is None:
            raise GameLogicError('next player is None')

        game.active_phase_player = next_player
        game.active_turn_player = next_player
        game.phase = GameManager.ROLL_THE_DICES
        game.save()

        if game.next_phase_task_id:
            app.control.revoke(game.next_phase_task_id, terminate=True)
            game.next_phase_task_id = None

        from .tasks import kick_out_callback

        #task = kick_out_callback.apply_async(args=[game.pk, next_player.pk], countdown=50)
        #game.kick_out_task_id = task.id

        game.save()
```

```
@classmethod
def _propose_trade(cls, game: Game, user: CustomUser, action: ActionTradeProposal) -> None:
    if action.player != user or action.offered_money < 0 or action.asked_money < 0:
        raise MaliciousUserInput(user, "cannot do operation")
    if action.destination_user not in game.players.all():
        # FIXME: Change to internal order so that it handles player change to AI
        raise MaliciousUserInput(user, "referenced a player that is not in game")

    asked_properties_list = action.asked_properties.all()
    asked_count = PropertyRelationship.objects.filter(
        game=game,
        owner=action.destination_user,
        id__in=action.asked_properties.all()
    ).count()

    if asked_count != asked_properties_list.count():
        raise MaliciousUserInput(user, "destination does not have enough properties")

    offered_properties_list = action.offered_properties.all()
    offered_count = PropertyRelationship.objects.filter(
        game=game,
        owner=action.player,
        id__in=offered_properties_list
    ).count()

    if offered_count != offered_properties_list.count():
        raise MaliciousUserInput(user, "offer does not have enough properties")

    all_trade_properties = list(asked_properties_list) + list(offered_properties_list)
    for rel in all_trade_properties:
        real_sq = rel.square.get_real_instance()
        if isinstance(real_sq, PropertySquare):
            group_has_houses = PropertyRelationship.objects.filter(
                game=game,
                square__propertysquare__group=real_sq.group,
                houses__gt=0
            ).exists()
            if group_has_houses:
                raise MaliciousUserInput(user, "cannot trade properties from a group with constructions")

    game.phase = GameManager.PROPOSAL_ACCEPTANCE
    game.active_phase_player = action.destination_user
    game.proposal = action # type: ignore
    game.save()

@classmethod
def _bankrupt_player(cls, game: Game, user: CustomUser) -> None:
    properties = PropertyRelationship.objects.filter(game=game, owner=user)

    #reset all properties
    for rel in properties:
        rel.owner = None
        rel.houses = -1
        rel.mortgage = False
        rel.save()


    game.money[str(user.pk)] = 0
```

```
        # next_turns fails if we dont do this sh
        game.ordered_players = [pk for pk in game.ordered_players if pk != user.pk]
        game.save()

        if game.active_turn_player == user:
            cls._next_turn(game, user)

        game.players.remove(user)
        game.save()

        # TODO: if only one left -> he wins

    #TODO: llegar a fase final donde se reparte esto
    @classmethod
    def _apply_end_bonuses(cls, game: Game, num_bonuses: int = 3) -> ResponseBonus:
        all_categories = list(BonusCategory.objects.all())
        chosen = random.sample(all_categories, min(num_bonuses, len(all_categories)))

        response = ResponseBonus()
        bonuses = {}

        for category in chosen:
            field = category.stat_field
            stats = PlayerGameStatistic.objects.filter(game=game)

            max_value = stats.aggregate(Max(field))[f'{field}__max']
            if max_value and max_value > 0:
                winners = list(stats.filter(**{field: max_value}).values_list('user__pk', flat=True))
                for pk in winners:
                    game.money[str(pk)] += category.bonus_amount
            else:
                winners = []

            bonuses[str(category.pk)] = {
                'bonus_amount': category.bonus_amount,
                'winners': winners
            }

        response.bonuses = bonuses
        game.save()
        return response

    @classmethod
    def _end_game_logic(cls, game: Game, user: CustomUser, action: Action) -> Response:
        if not game.finished:
            game.finished = True
            response = cls._apply_end_bonuses(game, num_bonuses=3)
            response.save()
            game.bonus_response = response
            game.save()
            return response
        else:
            raise GameLogicError('game was already ended')
```

`process_action(game, user, action)` `classmethod`

The only public method exposed in the API. It processes each action in dedicated functions depending on the current phase and returns a Response.

Parameters:

Name	Type	Description	Default
`game`	`Game`	The current active game instance.	*required*
`user`	`CustomUser`	The user performing the action.	*required*
`action`	`Action`	The deserialized action payload.	*required*

Returns:

Name	Type	Description
`Response`	`Response`	A response object representing the outcome.

Raises:

Type	Description
`MaliciousUserInput`	If the user acts out of turn or phase.
`GameLogicError`	If an unrecognized phase is encountered.

- **Source code in** `magnate/games.py`

```
@classmethod
@database_sync_to_async
def process_action(cls, game: Game, user: CustomUser, action: Action) -> Response:
    """
    The only public method exposed in the API. It processes each action
    in dedicated functions depending on the current phase and returns
    a Response.

    Args:
        game (Game): The current active game instance.
        user (CustomUser): The user performing the action.
        action (Action): The deserialized action payload.

    Returns:
        Response: A response object representing the outcome.

    Raises:
        MaliciousUserInput: If the user acts out of turn or phase.
        GameLogicError: If an unrecognized phase is encountered.
    """
    if isinstance(action, ActionSurrender):
        # TODO
        pass

    if user != game.active_phase_player and not isinstance(action, ActionBid): # if aucction there are no turns
        raise MaliciousUserInput(user, "is not the active player")

    if game.phase == cls.ROLL_THE_DICES:
        if isinstance(action, ActionPayBail):
            response = cls._pay_bail_logic(game, user, action)
        elif isinstance(action, ActionThrowDices):
            response = cls._roll_dices_logic(game, user, action)
        else:
```

```
            raise MaliciousUserInputAction(game, user, action)
    elif game.phase == cls.CHOOSE_SQUARE:
        if not isinstance(action, ActionMoveTo):
            raise MaliciousUserInputAction(game, user, action)
        response = cls._square_chosen_logic(game, user, action)
    elif game.phase == cls.CHOOSE_FANTASY:
        if not isinstance(action, ActionChooseCard):
            raise MaliciousUserInputAction(game, user, action)
        response = cls._choose_fantasy_logic(game, user, action)
    elif game.phase == cls.MANAGEMENT:
        if not isinstance(action, (ActionBuySquare, ActionDropPurchase, ActionTakeTram, ActionNextPhase)):
            raise MaliciousUserInputAction(game, user, action)
        response = cls._management_logic(game, user, action)
    elif game.phase == cls.BUSINESS or game.phase == cls.LIQUIDATION:
        if not isinstance(action, (ActionBuild, ActionDemolish, ActionTradeProposal, ActionMortgageSet, ActionMortgag
            raise MaliciousUserInputAction(game, user, action)
        response = cls._business_logic(game, user, action)
    elif game.phase == cls.ANSWER_TRADE_PROPOSAL:
        if not isinstance(action, ActionTradeAnswer):
            raise MaliciousUserInputAction(game, user, action)
        response = cls._answer_trade_proposal_logic(game, user, action)
    elif game.phase == cls.AUCTION:
        if not isinstance(action, ActionBid):
            raise MaliciousUserInputAction(game, user, action)
        response = cls._bid_property_auction_logic(game, user, action)
    elif game.phase == cls.END_GAME:
        # TODO: check instance???
        # response = cls._end_game_logic(game,user,action)
        return cls._end_game_logic(game,user,action)
    else:
        raise GameLogicError(f"Fase no reconocida o no manejada: {game.phase}")

    return _add_basic_response_data(game, response)
```

`_pay_bail_logic(game, user, action)` `staticmethod`

Processes the action to pay the jail bail.

Parameters:

Name	Type	Description	Default
`game`	`Game`	The current game instance.	*required*
`user`	`CustomUser`	The jailed user.	*required*
`action`	`ActionPayBail`	The bail action payload.	*required*

Returns:

Name	Type	Description
`Response`	`Response`	The standard response.

Raises:

Type	Description
`MaliciousUserInput`	If the user is not actually in jail.
`GameLogicError`	If the user does not have enough money.

- **Source code in** `magnate/games.py`

```
@staticmethod
def _pay_bail_logic(game: Game, user: CustomUser, action: ActionPayBail) -> Response:
    """
    Processes the action to pay the jail bail.

    Args:
        game (Game): The current game instance.
        user (CustomUser): The jailed user.
        action (ActionPayBail): The bail action payload.

    Returns:
        Response: The standard response.

    Raises:
        MaliciousUserInput: If the user is not actually in jail.
        GameLogicError: If the user does not have enough money.
    """
    square = _get_user_square(game, user).get_real_instance()

    if not isinstance(square, JailSquare):
        raise MaliciousUserInput(user, "is not in jail")

    remaining = game.jail_remaining_turns.get(str(user.pk), 0)
    if remaining == 0:
        raise MaliciousUserInput(user, "is not in jail (no turns remaining)")

    bail_price = square.bail_price

    if game.money[str(user.pk)] < bail_price:
        raise GameLogicError("not enough money to pay bail")

    game.money[str(user.pk)] -= bail_price
    game.jail_remaining_turns[str(user.pk)] = 0
    game.save()

    stats = PlayerGameStatistic.objects.get(user=user,game=game)
    stats.lost_money += bail_price
    stats.save()
    # roll the dices -> continue normally

    return Response()
```

`_roll_dices_logic(game, user, action)` `staticmethod`

Handles rolling the dice, checking for doubles/triples, and resolving jail mechanics.

Parameters:

Name	Type	Description	Default
`game`	`Game`	The current game instance.	*required*
`user`	`CustomUser`	The user throwing the dice.	*required*
`action`	`ActionThrowDices`	The dice throw action payload.	*required*

Returns:

Name	Type	Description
`Response`	`Response`	Standard response.

- **Source code in** `magnate/games.py`

```
@staticmethod
def _roll_dices_logic(game: Game, user: CustomUser, action: ActionThrowDices) -> Response:
    """
    Handles rolling the dice, checking for doubles/triples, and resolving jail mechanics.

    Args:
        game (Game): The current game instance.
        user (CustomUser): The user throwing the dice.
        action (ActionThrowDices): The dice throw action payload.

    Returns:
        Response: Standard response.
    """
    response: ResponseThrowDices = ResponseThrowDices()

    d1 = random.randint(1,6)
    d2 = random.randint(1,6)
    d3 = random.randint(1,6) # 4-6 are the bus faces

    response.dice1, response.dice2, response.dice_bus = d1, d2, d3

    triples = d3 <= 3 and (d1 == d2 == d3)
    doubles = (d1 == d2) and not triples

    response.triple = triples

    current_pos_square = _get_user_square(game, user).get_real_instance()
    current_pos_id = current_pos_square.custom_id

    # jail logic
    remaining_jail_turns = game.jail_remaining_turns.get(str(user.pk), 0)
    is_jailed = remaining_jail_turns > 0

    if is_jailed:
        jail_sq = current_pos_square
        if isinstance(jail_sq, JailSquare):
            if remaining_jail_turns == 1: #obligado a salir
                game.money[str(user.pk)] -= jail_sq.bail_price
                game.jail_remaining_turns[str(user.pk)] = 0
                stats = PlayerGameStatistic.objects.get(user=user,game=game)
                stats.turns_in_jail += 1
                stats.lost_money += jail_sq.bail_price
                stats.save()
            elif doubles: #sale gratis
                game.jail_remaining_turns[str(user.pk)] = 0
                game.streak = 0
            else:
                # stays in jail
                game.jail_remaining_turns[str(user.pk)] -= 1
                game.phase = GameManager.BUSINESS
                game.save()
                stats = PlayerGameStatistic.objects.get(user=user,game=game)
```

```
            stats.turns_in_jail += 1
            stats.save()
            response.path = [current_pos_id]
            return response
    else:
        raise GameLogicError(f"Cannot be in jail status and not in jail square")

# Not jailed
if triples:
    # path current -> decided in chosen
    response.triple = True
    square = current_pos_square
    all_squares = BaseSquare.objects.filter(board=square.board)
    # All squares are suitable destinations
    possible_destinations = [s.custom_id for s in all_squares]
    possible_destinations.remove(_get_jail_square().custom_id)
    game.possible_destinations = {str(c_id): 0 for c_id in possible_destinations}
    response.destinations = possible_destinations
    game.phase = GameManager.CHOOSE_SQUARE
    response.path = [current_pos_id]
    game.save()
    return response
elif doubles: # doubles streak only if not getting out of jail via doubles
    if game.streak >= 2:
        # path -> current and jail
        jail_square = _get_jail_square()
        response.destinations = [jail_square.custom_id]
        game.streak = 0
        response.streak = game.streak

        game.positions[str(user.pk)] = jail_square.custom_id
        game.jail_remaining_turns[str(user.pk)] = 3
        response.path = [current_pos_id, jail_square.custom_id]

        stats = PlayerGameStatistic.objects.get(user=user,game=game)
        stats.times_in_jail += 1
        stats.save()

        game.phase = GameManager.LIQUIDATION

        game.save()
        return response
    elif not is_jailed:
        game.streak = game.streak + 1
else:
    game.streak = 0

response.streak = game.streak

# Hasn't gone to jail
dice_combinations = _compute_dice_combinations(d1, d2, d3)
game.possible_destinations, passed_go_map = _get_possible_destinations_ids(game, user, dice_combinations)

response.destinations = list(game.possible_destinations.keys())
if len(game.possible_destinations) > 1:
    # path in square chosen logic
    game.phase = GameManager.CHOOSE_SQUARE
    response.path = [current_pos_id]
else:
```

```
        stats = PlayerGameStatistic.objects.get(user=user,game=game)
        stats.walked_squares += dice_combinations[0]
        stats.save()
        dest_square_id = next(iter(game.possible_destinations))
        steps = game.possible_destinations[dest_square_id]

        # Use _move_player_logic to get the traversed path and check for "Go to Jail" or "Passing Go".
        move_result = _move_player_logic(current_pos_square, steps)
        response.path = move_result["path"]

        if move_result["jailed"]:
            # landing in go to jail; update state to jail the player.
            game.positions[str(user.pk)] = move_result["final_id"]
            game.jail_remaining_turns[str(user.pk)] = 3
            game.phase = GameManager.LIQUIDATION
            stats = PlayerGameStatistic.objects.get(user=user, game=game)
            stats.times_in_jail += 1
            stats.save()
        else:
            game.positions[str(user.pk)] = dest_square_id
            square = _get_square_by_custom_id(dest_square_id)
            _apply_square_arrival(game, user, response, square, move_result["passed_go"])


    game.save()
    return response
```

`_square_chosen_logic(game, user, action)` `staticmethod`

Handles the logic when a user selects their final destination (if multiple choices were given).

Parameters:

Name	Type	Description	Default
`game`	`Game`	The current game instance.	*required*
`user`	`CustomUser`	The current user.	*required*
`action`	`ActionMoveTo`	Action indicating the chosen square.	*required*

Returns:

Name	Type	Description
`Response`	`Response`	Standard response.

Raises:

Type	Description
`MaliciousUserInput`	If the chosen square is not in `possible_destinations`.

- **Source code in** `magnate/games.py`

```
@staticmethod
def _square_chosen_logic(game: Game, user: CustomUser, action: Action) -> Response:
    """
    Handles the logic when a user selects their final destination (if multiple choices were given).

    Args:
```

```
        game (Game): The current game instance.
        user (CustomUser): The current user.
        action (ActionMoveTo): Action indicating the chosen square.

    Returns:
        Response: Standard response.

    Raises:
        MaliciousUserInput: If the chosen square is not in `possible_destinations`.
    """
    response: ResponseChooseSquare = ResponseChooseSquare()

    if not isinstance(action, ActionMoveTo):
        raise MaliciousUserInputAction(game, user, action)

    square = action.square

    if str(square.custom_id) not in game.possible_destinations:
        raise MaliciousUserInput(user, "tried to move to an illegal square")

    current_pos_id = game.positions[str(user.pk)]
    current_pos_square = _get_square_by_custom_id(current_pos_id).get_real_instance()

    steps = game.possible_destinations.get(str(square.custom_id), 0)

    move_result = _move_player_logic(current_pos_square, steps)

    if steps == 0:
        response.path = [current_pos_id, square.custom_id]
    else:
        response.path = move_result["path"]

    game.positions[str(user.pk)] = square.custom_id

    stats = PlayerGameStatistic.objects.get(user=user,game=game)
    stats.walked_squares += steps
    stats.save()

    game.possible_destinations = dict()

    # Use the passed_go flag correctly computed by _move_player_logic.
    _apply_square_arrival(game, user, response, action.square, move_result["passed_go"])


    game.save()
    return response
```

`_management_logic(game, user, action)` `staticmethod`

Logic of management phase, where the user can buy properties, pay bills etc.

It checks the user's action against the current square they are on to update the game state and transition to the next phase.

Parameters:

Name	Type	Description	Default
`game`	`Game`	The current game instance.	*required*
`user`	`CustomUser`	The acting user.	*required*
`action`	`Action`	The payload of the action taken (e.g., ActionBuySquare).	*required*

Returns:

Name	Type	Description
`Response`	`Response`	Standard response.

Raises:

Type	Description
`MaliciousUserInputAction`	If the action does not fit the phase/context.

- **Source code in** `magnate/games.py`

```
@staticmethod
def _management_logic(game: Game, user: CustomUser, action: Action) -> Response:
    """
    Logic of management phase, where the user can buy properties, pay bills etc.

    It checks the user's action against the current square they are on to update
    the game state and transition to the next phase.

    Args:
        game (Game): The current game instance.
        user (CustomUser): The acting user.
        action (Action): The payload of the action taken (e.g., ActionBuySquare).

    Returns:
        Response: Standard response.

    Raises:
        MaliciousUserInputAction: If the action does not fit the phase/context.
    """
    current_square = _get_user_square(game, user).get_real_instance()
    prop_rel = _get_relationship(game, current_square)

    if isinstance(action, ActionBuySquare):
        if isinstance(current_square, PropertySquare):
            # TODO: Check money
            game.money[str(user.pk)] -= current_square.buy_price
            stats = PlayerGameStatistic.objects.get(user=user,game=game)
            stats.lost_money += current_square.buy_price
            stats.save()
            new_property = PropertyRelationship(game=game, square=current_square, owner=user)
            user_properties = PropertyRelationship.objects.filter(game=game, owner=user)
            user_same_group_properties = user_properties.filter(square__propertysquare__group=current_square.group)

            group_squares = PropertySquare.objects.filter(group=current_square.group, board = current_square.board)

            if user_same_group_properties.count() == group_squares.count() - 1:
                new_property.houses = 0
                user_same_group_properties.update(houses=0)
            else:
```

```
            new_property.houses = -1
        new_property.save()

    elif isinstance(current_square, ServerSquare):
        game.money[str(user.pk)] -= current_square.buy_price
        stats = PlayerGameStatistic.objects.get(user=user,game=game)
        stats.lost_money += current_square.buy_price
        stats.save()
        new_property = PropertyRelationship(game=game, square=current_square, owner=user)
        new_property.save()

    elif isinstance(current_square, BridgeSquare):
        game.money[str(user.pk)] -= current_square.buy_price
        stats = PlayerGameStatistic.objects.get(user=user, game=game)
        stats.lost_money += current_square.buy_price
        stats.save()
        new_property = PropertyRelationship(game=game, square=current_square, owner=user)
        new_property.save()

    else:
        raise MaliciousUserInputAction(game, user, action)
elif isinstance(action, ActionDropPurchase):
    if isinstance(action.square, (PropertySquare, ServerSquare, BridgeSquare)):
        GameManager._initiate_auction(game, action.square)
    else:
        raise MaliciousUserInputAction(game, user, action)
elif isinstance(action, ActionTakeTram):
    if isinstance(current_square, TramSquare):
        square = action.square
        tram_squares = TramSquare.objects.filter()
        tram_square_actual_id = game.positions[str(user.pk)]
        tram_squares_extern_ids = [s.custom_id for s in tram_squares]

        if square.custom_id == tram_square_actual_id: # case stay in the same square, free
            pass
        elif square.custom_id in tram_squares_extern_ids: # Move to another square
            if game.money[str(user.pk)] < square.buy_price:
                raise MaliciousUserInput(user, "does not have enough money to take tram")
            game.money[str(user.pk)] -= square.buy_price
            stats = PlayerGameStatistic.objects.get(user=user,game=game)
            stats.lost_money += square.buy_price
            stats.save()
            game.positions[str(user.pk)] = square.custom_id
        else:
            raise MaliciousUserInput(user, "tried to take a tram to a non tram square")
    else:
        raise MaliciousUserInputAction(game, user, action)
elif isinstance(action, ActionNextPhase):
    # TODO: Remove?
    pass
else:
    raise MaliciousUserInputAction(game, user, action)

if game.phase == GameManager.MANAGEMENT:
    if game.streak == 0:
        game.phase = GameManager.BUSINESS
    else:
        game.phase = GameManager.ROLL_THE_DICES
```

```
    game.save()
    return Response()
```

`_business_logic(game, user, action)` `staticmethod`

Unifies the business and liquidation phases. Handles building, demolishing, trading, mortgaging, and proceeding to the next turn.

Parameters:

Name	Type	Description	Default
`game`	`Game`	The current game instance.	*required*
`user`	`CustomUser`	The active user.	*required*
`action`	`Action`	The payload of the action taken.	*required*

Returns:

Name	Type	Description
`Response`	`Response`	Standard response.

Raises:

Type	Description
`MaliciousUserInput`	If restrictions (e.g., negative balance on next turn) are unmet.

- **Source code in** `magnate/games.py`

```
@staticmethod
def _business_logic(game: Game, user: CustomUser, action: Action) -> Response:
    """
    Unifies the business and liquidation phases. Handles building, demolishing,
    trading, mortgaging, and proceeding to the next turn.

    Args:
        game (Game): The current game instance.
        user (CustomUser): The active user.
        action (Action): The payload of the action taken.

    Returns:
        Response: Standard response.

    Raises:
        MaliciousUserInput: If restrictions (e.g., negative balance on next turn) are unmet.
    """
    # Logic for the business phase where players can build houses, trade, etc.
    if isinstance(action, ActionBuild):
        # Check owner
        building_square = action.square
        relationship = _build_square(game, user, building_square, action.houses, False)

    elif isinstance(action, ActionDemolish):
        # Similiar to build
        demolition_square = action.square
        relationship = _demolish_square(game, user, demolition_square, action.houses, False)

    elif isinstance(action, ActionTradeProposal):
```

```
        relationship = GameManager._propose_trade(game, user, action)

    elif isinstance(action, ActionMortgageSet):
        relationship = _set_mortgage(game, user, action.square, False)

    elif isinstance(action, ActionMortgageUnset):
        relationship = _unset_mortgage(game, user, action.square, False)
    elif isinstance(action, ActionNextPhase):
        current_money = game.money[str(user.pk)]


        if game.phase == GameManager.BUSINESS:
            if current_money >= 0:
                GameManager._next_turn(game, user)
            else:
                game.phase = GameManager.LIQUIDATION
                game.save()

        elif game.phase == GameManager.LIQUIDATION:
            if current_money >= 0:
                GameManager._next_turn(game, user)
            else:
                raise MaliciousUserInput(user, "Cannot end in NEGATIVE")

    #TODO: elif with a timeout -> auto sell in case user can reach positive status or kick out in case

    return Response()
```